CSYE7374 Assignment1

Dataset

Kaggle: https://www.kaggle.com/datasets/starblasters8/human-vs-llm-text-corpus
The dataset consists of English sentences stored in CSV format with a single column text.
For this project, we considered the first **250,000 rows** to ensure manageable size for preprocessing and downstream experimentation.

Cleaning Strategies and Reasoning

- Removed duplicate documents to reduce redundancy and improve training signal.
- Normalized text by lowercasing, stripping whitespace, and removing irrelevant symbols, HTML tags, and reference markers.
- Filtered out short or low-quality documents (fewer than 50 words) to ensure meaningful training samples.
- Ensured uniform formatting for downstream tokenization and model ingestion.

Tokenization Choices

We used **Hugging Face AutoTokenizer** with **GPT-2 style Byte Pair Encoding (BPE)**.

- Vocabulary size: ~50,000 tokens.
- Tokenization method: subword-level (efficient for rare words).
- Block size: 128 tokens, chosen to balance batching efficiency and context length.
- Longer sequences were chunked into multiple blocks.

Data Loader Implementation Details

- Implemented a **custom PyTorch Dataset** to handle tokenized sequences.
- DataLoader supports **batching (batch size 8)** and **shuffling** for diverse training input.
- **Padding and truncation** ensure each sequence matches the chosen block size.
- **Iterable streaming** prevents memory bottlenecks when working with large datasets.

Challenges Encountered

- **Memory constraints** when handling large datasets, mitigated by chunking and streaming.
- **Lack of padding token** in GPT-2 tokenizer, solved by reassigning EOS token as PAD.

Reflections on Preprocessing Impact

Preprocessing significantly impacts the quality of model pretraining.

- Cleaning reduces noise and duplication, ensuring the model learns from diverse, high-quality data.
- Tokenization choices affect vocabulary coverage and efficiency.
- Proper batching and padding enable stable training without wasting resources.
- Overall, thoughtful preprocessing improves convergence speed, reduces overfitting, and enhances downstream task performance.

Conclusion

This project demonstrates the end-to-end design of a preprocessing pipeline for large-scale text data. By combining cleaning, tokenization, and efficient data loading strategies, we created a robust foundation for model pretraining. While challenges such as memory constraints and tokenizer limitations required creative solutions, the resulting dataset and pipeline are well-suited for future language modeling experiments.